

Università di Messina — CdL in Informatica — Reti e Sistemi Distribuiti Mod.B
Hands-On #18

Algoritmi distribuiti in C: shortest-path-tree e broadcast

Last Update: May 17, 2026

Scenario

In questo HO implementeremo in C i due algoritmi studiati a lezione — **Flooding** e **Bellman-Ford distribuito** — simulando fedelmente il modello di rete sincrona visto in aula.

La traduzione dal modello teorico al codice segue tre scelte di design precise:

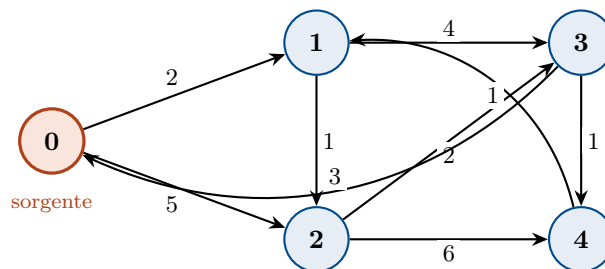
- **Un processo per ogni processore.** Ogni nodo della rete è un processo Unix creato con `fork()`. I processi non condividono memoria, esattamente come nel modello distribuito.
- **Una pipe per ogni arco.** Per ogni arco $(i \rightarrow j)$ del grafo esiste una pipe anonima `msg_pipe[i][j]` su cui il processo i scrive i messaggi e il processo j li legge.
- **Sincronizzazione tramite pipe di ack.** La semantica sincrona — tutti i processori completano il round ℓ prima di iniziare il round $\ell + 1$ — emerge dalle `read()` bloccanti stesse, senza semafori né coordinatori esterni. Il meccanismo è spiegato nella Sezione 4.

Il programma esegue i due algoritmi in sequenza sullo stesso grafo e stampa il risultato di ciascuno. I file da consegnare sono **un unico sorgente C** compilabile con:

```
gcc -Wall -Wextra -o ho18 ho18.c -lm && ./ho18
```

La rete

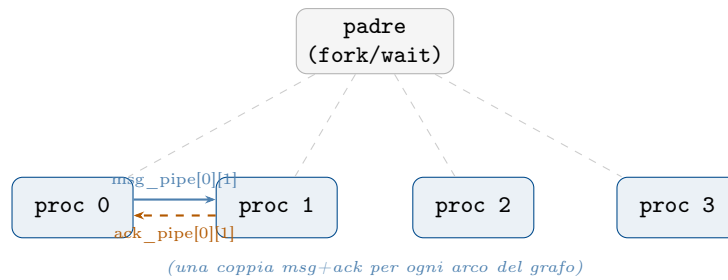
Il grafo su cui lavorerete ha **5 nodi** (numerati da 0 a 4) e **9 archi diretti** con pesi strettamente positivi. Il nodo **0** è la sorgente per entrambi gli algoritmi.



Architettura del simulatore

Processi e pipe

Il padre chiama `create_pipes()` e poi `fork()` N volte. Ogni figlio riceve l'indice `id` $\in \{0, \dots, N-1\}$, chiude i file descriptor che non usa, ed esegue `run_flooding(id)` o `run_bf(id)`.



Per ogni arco ($i \rightarrow j$) del grafo esistono **due pipe**:

Pipe	Chi scrive	Chi legge
<code>msg_pipe[i][j]</code>	processo i (lato [1])	processo j (lato [0])
<code>ack_pipe[i][j]</code>	processo j (lato [1])	processo i (lato [0])

`ack_pipe[i][j]` scorre in direzione opposta a `msg_pipe[i][j]`: dopo aver letto il messaggio di i , il processo j invia un ack a i per segnalare che la pipe è pronta per il round successivo.

Il meccanismo di sincronizzazione

Ogni round è diviso in **quattro fasi** eseguite da tutti i processi nello stesso ordine:

Fase 1	MSG	ogni processo scrive il proprio messaggio su <code>msg_pipe[id][j]</code> per ogni out-neighbor j
Fase 2	STF	ogni processo legge da <code>msg_pipe[i][id]</code> per ogni in-neighbor i , poi chiama <code>stf()</code>
Fase 3	ACK	ogni processo scrive un ack su <code>ack_pipe[i][id]</code> per ogni in-neighbor i (“ho letto il tuo messaggio”)
Fase 4	WAIT	ogni processo legge un ack da <code>ack_pipe[id][j]</code> per ogni out-neighbor j (attende che tutti abbiano letto)

La sincronizzazione emerge automaticamente dalle `read()` bloccanti: la Fase 2 del processo j non può avanzare finché il processo i non ha completato la Fase 1 (la `read()` su `msg_pipe[i][j]` blocca j fino a quando i non scrive). La Fase 4 impedisce che i sovrascriva la pipe con i messaggi del round $\ell+1$ prima che j abbia consumato quelli del round ℓ .

► Perché servono entrambe le pipe?

Senza la pipe di ack, il processo i potrebbe scrivere il messaggio del round $\ell+1$ su `msg_pipe[i][j]` prima che j abbia letto quello del round ℓ , corrompendo la comunicazione. La `read()` bloccante in Fase 4 garantisce che i parta solo quando tutti i suoi out-neighbor hanno effettivamente consumato il messaggio corrente.

Il codice fornito

Trovate nel file `ho18_skeleton.c` le seguenti funzioni già implementate. Leggetele con attenzione prima di iniziare i task.

`create_pipes()` — crea tutte le pipe

Listing 1: `create_pipes()`: alloca NxN coppie di pipe

```
1 void create_pipes(void) {
2     for (int i = 0; i < N; i++)
```

```

3         for (int j = 0; j < N; j++) {
4             if (i == j) continue;           /* niente self-loop */
5             if (pipe(msg_pipe[i][j]) < 0) { perror("pipe msg"); exit(1); }
6             if (pipe(ack_pipe[i][j]) < 0) { perror("pipe ack"); exit(1); }
7         }
8     }

```

Crea $N \times (N-1)$ coppie di pipe, una per ogni possibile arco diretto. Le pipe vengono create per tutti i potenziali archi, non solo per quelli presenti nel grafo: le pipe inutilizzate vengono semplicemente chiuse da `close_unused_fds()`.

`close_unused_fds()` — ogni figlio chiude i capi che non usa

Listing 2: `close_unused_fds(id)`: gestione dei descrittori dopo `fork()`

```

1 void close_unused_fds(int id) {
2     for (int i = 0; i < N; i++) {
3         for (int j = 0; j < N; j++) {
4             if (i == j) continue;
5             if (i == id) {
6                 /* id e' il mittente su msg_pipe[id][j]:
7                  * tiene il lato scrittura [1], chiude il lato lettura
8                  * [0].
9                  * id e' il destinatario di ack_pipe[id][j]:
10                 * tiene il lato lettura [0], chiude il lato scrittura
11                 * [1]. */
12                 close(msg_pipe[id][j][0]);
13                 close(ack_pipe[id][j][1]);
14             } else if (j == id) {
15                 /* id e' il destinatario su msg_pipe[i][id]:
16                 * tiene il lato lettura [0], chiude il lato scrittura
17                 * [1].
18                 * id e' il mittente di ack su ack_pipe[i][id]:
19                 * tiene il lato scrittura [1], chiude il lato lettura
20                 * [0]. */
21                 close(msg_pipe[i][id][1]);
22                 close(ack_pipe[i][id][0]);
23             } else {
24                 /* pipe tra altri due processi: id chiude entrambi i lati
25                 */
26                 close(msg_pipe[i][j][0]); close(msg_pipe[i][j][1]);
27                 close(ack_pipe[i][j][0]); close(ack_pipe[i][j][1]);
28             }
29         }
30     }
31 }

```

[!] Dimenticare di chiudere i capi inutilizzati

Se un processo tiene aperto il lato di *scrittura* di una pipe da cui non scriverà mai, la `read()` dell'altro processo non restituirà mai EOF e si bloccherà per sempre. Questo è il bug più comune con le pipe in C. `close_unused_fds()` lo previene chiudendo sistematicamente ogni file descriptor non necessario.

`send_double()`, `recv_double()`, `send_ack()`, `recv_ack()`

Listing 3: Primitive di I/O sulle pipe

```

1 void send_double(int fd, double val) {
2     /* Scrive sizeof(double) = 8 byte atomicamente sulla pipe.

```

```

3      * Le pipe Linux garantiscono atomicita' per scritture <= PIPE_BUF
      (4096 B). */
4      if (write(fd, &val, sizeof(double)) != sizeof(double))
5          { perror("write"); exit(1); }
6  }
7
8  double recv_double(int fd) {
9      double val;
10     /* Blocca finche' il mittente non ha scritto 8 byte. */
11     if (read(fd, &val, sizeof(double)) != sizeof(double))
12         { perror("read"); exit(1); }
13     return val;
14 }
15
16 void send_ack(int fd) {
17     char c = 1;
18     write(fd, &c, 1); /* 1 byte e' sufficiente come segnale di
      sincronizzazione */
19 }
20
21 void recv_ack(int fd) {
22     char c;
23     read(fd, &c, 1); /* blocca finche' l'ack non arriva */
24 }

```

I messaggi sono trasmessi come valori `double` a 8 byte in formato binario nativo, senza serializzazione. Il valore sentinella `NULL_MSG = -1.0` segnala l'assenza di messaggio (tutti i pesi sono positivi, quindi `-1` è inequivocabilmente una sentinella).

`run_flooding()` e `run_bf()` — il loop dei round

Queste funzioni implementano il loop dei round per i due algoritmi. La struttura è identica: le quattro fasi si ripetono per `ROUNDS` round. I dettagli dell'algoritmo sono delegati alle funzioni `msg` e `stf` che voi implementerete.

Listing 4: `run_flooding()`: loop dei round con le quattro fasi

```

1  void run_flooding(int id) {
2      /* Stato iniziale secondo il modello formale */
3      FloodState s;
4      s.parent = (id == SOURCE) ? SOURCE : NO_PAR;
5      s.data = (id == SOURCE) ? 1 : 0; /* 1 = ha il token */
6      s.snd_flag = (id == SOURCE) ? 1 : 0;
7
8      for (int r = 0; r < ROUNDS; r++) {
9          /* Fase 1: ogni out-neighbor riceve il risultato di flood_msg() */
10         for (int j = 0; j < N; j++) {
11             if (j == id || adj[id][j] == 0.0) continue;
12             send_double(msg_pipe[id][j][1], flood_msg(id, &s, j));
13         }
14         /* Fase 2: raccogli messaggi dagli in-neighbor, poi aggiorna stato
           */
15         double in_msgs[N];
16         for (int i = 0; i < N; i++) in_msgs[i] = NULL_MSG;
17         for (int i = 0; i < N; i++) {
18             if (i == id || adj[i][id] == 0.0) continue;
19             in_msgs[i] = recv_double(msg_pipe[i][id][0]); /* bloccante */
20         }
21         flood_stf(id, &s, in_msgs);
22         /* Fase 3: ack verso ogni in-neighbor ("ho letto il tuo messaggio
           ") */
23         for (int i = 0; i < N; i++) {
24             if (i == id || adj[i][id] == 0.0) continue;

```

```

25         send_ack(ack_pipe[i][id][1]);
26     }
27     /* Fase 4: attendi ack da ogni out-neighbor prima del round
        successivo */
28     for (int j = 0; j < N; j++) {
29         if (j == id || adj[id][j] == 0.0) continue;
30         recv_ack(ack_pipe[id][j][0]);
31     }
32 }
33 printf("[FLOOD] nodo %d: parent=%2d token=%s\n",
34        id, s.parent, s.data ? "SI" : "NO");
35 fflush(stdout);
36 }

```

`run_bf()` ha la stessa struttura: state iniziale diverso (`dist = 0` per la sorgente, `INF` per gli altri) e chiama `bf_msg()` e `bf_stf()` al posto delle controparti per il flooding.

main() — fork, wait e sequenziamento

Listing 5: `main()`: esegue flooding poi Bellman-Ford

```

1  int main(void) {
2      /* ?? Parte 1: Flooding ?????????????????????????????????????????? */
3      printf("== Flooding Algorithm (sorgente: nodo %d) ==\n", SOURCE);
4      create_pipes();
5      for (int id = 0; id < N; id++) {
6          pid_t pid = fork();
7          if (pid < 0) { perror("fork"); exit(1); }
8          if (pid == 0) { /* figlio */
9              close_unused_fds(id);
10             run_flooding(id);
11             exit(0);
12         }
13     }
14     /* Il padre chiude tutte le sue copie delle pipe e aspetta i figli */
15     for (int i = 0; i < N; i++)
16         for (int j = 0; j < N; j++) {
17             if (i == j) continue;
18             close(msg_pipe[i][j][0]); close(msg_pipe[i][j][1]);
19             close(ack_pipe[i][j][0]); close(ack_pipe[i][j][1]);
20         }
21     for (int i = 0; i < N; i++) wait(NULL);
22
23     /* ?? Parte 2: Bellman-Ford ?????????????????????????????????????????? */
24     printf("\n== Distributed Bellman-Ford (sorgente: nodo %d) ==\n",
25           SOURCE);
26     create_pipes();
27     for (int id = 0; id < N; id++) {
28         pid_t pid = fork();
29         if (pid < 0) { perror("fork"); exit(1); }
30         if (pid == 0) {
31             close_unused_fds(id);
32             run_bf(id);
33             exit(0);
34         }
35     }
36     for (int i = 0; i < N; i++)
37         for (int j = 0; j < N; j++) {
38             if (i == j) continue;
39             close(msg_pipe[i][j][0]); close(msg_pipe[i][j][1]);
40             close(ack_pipe[i][j][0]); close(ack_pipe[i][j][1]);
41         }
42     for (int i = 0; i < N; i++) wait(NULL);

```

```

42     return 0;
43 }

```

▷ Perché il padre chiude tutte le pipe?

Dopo il `fork()`, il padre ha ereditato tutte le copie dei file descriptor. Se non le chiudesse, i lati di scrittura resterebbero aperti anche dopo che tutti i figli hanno terminato: le `read()` dei figli non riceverebbero mai EOF e il programma si bloccherebbe.

Task 1 — Flooding Algorithm

1.1 — Topologia: la matrice di adiacenza

Task 1.1 — Definire la matrice di adiacenza

Completate la definizione di `adj[N][N]` in `ho18.c` in modo da rappresentare il grafo della Sezione 2. Il valore `adj[i][j]` deve essere il peso dell'arco $i \rightarrow j$, oppure `0.0` se l'arco non esiste. Non esistono self-loop (`adj[i][i] = 0.0`).

Elenco degli archi: $0 \rightarrow 1:2$, $0 \rightarrow 2:5$, $1 \rightarrow 2:1$, $1 \rightarrow 3:4$, $2 \rightarrow 3:2$, $2 \rightarrow 4:6$, $3 \rightarrow 4:1$, $3 \rightarrow 0:3$, $4 \rightarrow 1:1$.

1.2 — La funzione `flood_msg()`

Nel modello formale, `msg(w, i)` restituisce il messaggio da inviare al vicino i dato lo stato corrente $w = (\text{parent}, \text{data}, \text{snd_flag})$. Per il Flooding:

- se `snd_flag = 1` e il vicino i non è il `parent`, restituisce `1.0` (trasmette il token);
- altrimenti restituisce `NULL_MSG`.

Il nodo non ritrasmette verso il proprio parent per non creare cicli nell'albero BFS. La sorgente ha `parent = SOURCE = 0`, quindi trasmette a tutti i suoi out-neighbor.

Task 1.2 — Implementare `flood_msg()`

```

1  /* Restituisce il messaggio da inviare al vicino 'neighbor'.
2  * Usate NULL_MSG (-1.0) per indicare "nessun messaggio". */
3  double flood_msg(int id, FloodState *s, int neighbor) {
4      /* DA IMPLEMENTARE */
5  }

```

1.3 — La funzione `flood_stf()`

`stf(w, y)` riceve il vettore dei messaggi `in_msgs[N]` (dove `in_msgs[i] = NULL_MSG` se i non è un in-neighbor o non ha inviato nulla) e aggiorna lo stato del processore. I tre casi del modello formale:

1. `data = 0` e **tutti** `NULL_MSG`: il token non è ancora arrivato; lo stato rimane invariato.
2. `data = 0` e **almeno un** `msg` $\neq$ `NULL_MSG`: token ricevuto per la prima volta. Il `parent` diventa il mittente con **UID minore** tra tutti quelli che hanno inviato un messaggio non-null. Si pone `data = 1` e `snd_flag = 1`.
3. `data = 1`: token già posseduto; si pone solo `snd_flag = 0` (non ritrasmettere più).

Task 1.3 — Implementare `flood_stf()`

```
1  /* Aggiorna lo stato in base ai messaggi ricevuti.
2   * in_msgs[i] == NULL_MSG se i non ha inviato nulla. */
3  void flood_stf(int id, FloodState *s, double in_msgs[N]) {
4      /* DA IMPLEMENTARE */
5  }
```

▷ Come trovare il parent con UID minore

Scorrete `in_msgs` da `i = 0` a `N-1`: il primo indice con `in_msgs[i] != NULL_MSG` che trovate ha già l'UID minore tra quelli che hanno inviato, perché state scorrendo in ordine crescente. In alternativa, inizializzate `new_parent = NO_PAR` e aggiornatelo ogni volta che trovate un mittente con UID minore di quello attuale.

Task 2 — Distributed Bellman-Ford

2.1 — La funzione `bf_msg()`

Nel Bellman-Ford distribuito, `msg(w, i)` è indipendente dal vicino `i`: ogni processore trasmette semplicemente la propria stima `dist` a tutti gli out-neighbor, per i primi `N` round. Dopo `N` round invia `NULL_MSG`. Poiché `run_bf()` esegue esattamente `ROUNDS = N` round, in pratica la condizione su `round` è sempre vera; va comunque codificata per fedeltà al modello formale.

Task 2.1 — Implementare `bf_msg()`

```
1  /* Restituisce il messaggio da inviare al vicino 'neighbor' al round '
   round'.
```

```
2   * Trasmette dist se round < N, altrimenti NULL_MSG. */
3  double bf_msg(int id, BFState *s, int round, int neighbor) {
4      /* DA IMPLEMENTARE */
5  }
```

2.2 — La funzione `bf_stf()`

`stf(w, y)` implementa il rilassamento degli archi distribuito. Per ogni in-neighbor `i` che ha inviato un valore non-`NULL_MSG`:

$$\text{candidato} = \text{in_msgs}[i] + \text{adj}[i][\text{id}]$$

Se `candidato < s->dist`, aggiornate `dist` e `parent`. Il processo compara *tutti* gli in-neighbor nel medesimo round e tiene il candidato migliore.

Task 2.2 — Implementare `bf_stf()`

```
1  /* Aggiorna dist e parent se un in-neighbor offre un percorso piu'
   corto.
```

```
2   * adj[i][id] e' il peso dell'arco i -> id. */
3  void bf_stf(int id, BFState *s, double in_msgs[N]) {
4      /* DA IMPLEMENTARE */
5  }
```

▷ Gestire correttamente INF

La sorgente inizia con `dist = 0.0`; tutti gli altri con `dist = INF = 1e18`. Nel primo round, ogni out-neighbor della sorgente riceve `0.0 + peso_arco` e aggiorna la propria stima. Verificate che `in_msgs[i] != NULL_MSG` prima di sommare il peso: un `NULL_MSG = -1.0` sommato a un peso darebbe un candidato negativo scorretto.

[!] Non aggiornare in-place durante la fase STF

`bf_stf()` viene chiamata *dopo* che tutti i messaggi del round corrente sono già stati ricevuti (Fase 2 è completata). Non dovete preoccuparvi di aggiornamenti in-place: ogni processore aggiorna solo il proprio stato locale `s->dist` e `s->parent`, che il processore stesso leggerà solo nel round successivo. La sincronizzazione garantisce che il round $r+1$ non inizi prima che tutti abbiano finito il round r .

Compilazione ed esecuzione

```
gcc -Wall -Wextra -o ho18 ho18.c -lm
./ho18
```

Output atteso (l'ordine delle righe può variare perché i processi sono paralleli):

```
=== Flooding Algorithm (sorgente: nodo 0) ===
[FL00D] nodo 0: parent= 0 token=SI
[FL00D] nodo 1: parent= 0 token=SI
[FL00D] nodo 2: parent= 0 token=SI
[FL00D] nodo 3: parent= 1 token=SI
[FL00D] nodo 4: parent= 2 token=SI

=== Distributed Bellman-Ford (sorgente: nodo 0) ===
[BF] nodo 0: dist= 0 parent= -
[BF] nodo 1: dist= 2 parent= 0
[BF] nodo 2: dist= 3 parent= 1
[BF] nodo 3: dist= 5 parent= 2
[BF] nodo 4: dist= 6 parent= 3
```

▷ Ordine non deterministico dell'output

I processi figli vengono schedulati dal kernel in ordine non deterministico. L'output del nodo 3 può apparire prima di quello del nodo 1 senza che il programma sia sbagliato. La correttezza si verifica sui valori, non sull'ordine delle righe.

Debug: cosa controllare se il programma si blocca

Se `./ho18` non termina entro pochi secondi, il programma è in deadlock. Le cause più comuni:

1. **Pipe con lato scrittura ancora aperto.** Un processo ha dimenticato di chiudere un lato di scrittura che non usa: la `read()` dell'altro processo attende un EOF che non arriverà mai. Verificate che `close_unused_fds()` venga chiamata all'inizio di ogni figlio e che il padre chiuda tutte le sue copie dopo i `fork()`.
2. **Asimmetria msg/ack.** Se un processo invia un ack senza che nessuno lo attenda, la corrispondente `recv_ack()` non verrà mai eseguita e il mittente del round successivo si bloccherà in Fase 4. Verificate che le fasi 3 e 4 iterino sulle stesse condizioni della rete (`adj[i][id] > 0` e `adj[id][j] > 0` rispettivamente).

3. **NULL_MSG** non gestito in `stf()`. Se `bf_stf()` aggiorna `dist` anche quando `in_msgs[i] == NULL_MSG`, il calcolo è scorretto ma il programma non si blocca. Controllate con `-fsanitize=address` eventuali accessi a pipe non inizializzate.